EXPERIMENT-1

TITLE: Multi-threaded Client/Server using Java RMI

AIM: To implement a multi-threaded client-server communication system using Java RMI, where multiple clients can connect and interact with the server simultaneously.

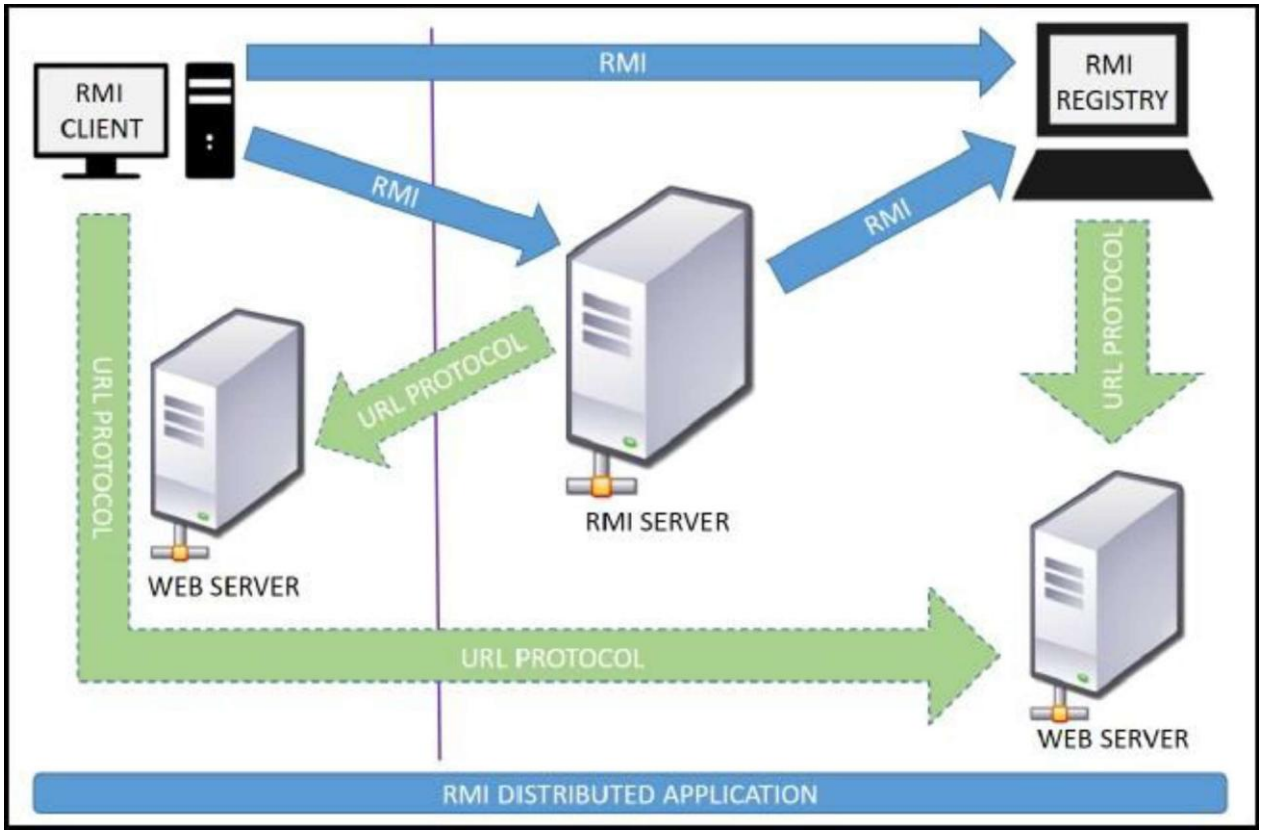
TOOLS: Java Programming Environment, jdk 1.8, rmiregistry

THEORY:

RMI :-

Java Remote Method Invocation (RMI) is a mechanism that allows an object running in one Java Virtual Machine (JVM) to invoke methods on an object running in another JVM. It is used to build distributed applications, where different components of a system communicate over a network.

The following diagram shows how RMI happens between the RMI client and RMI server with the help of the RMI registry :



In the RMI architecture, there are mainly three components:

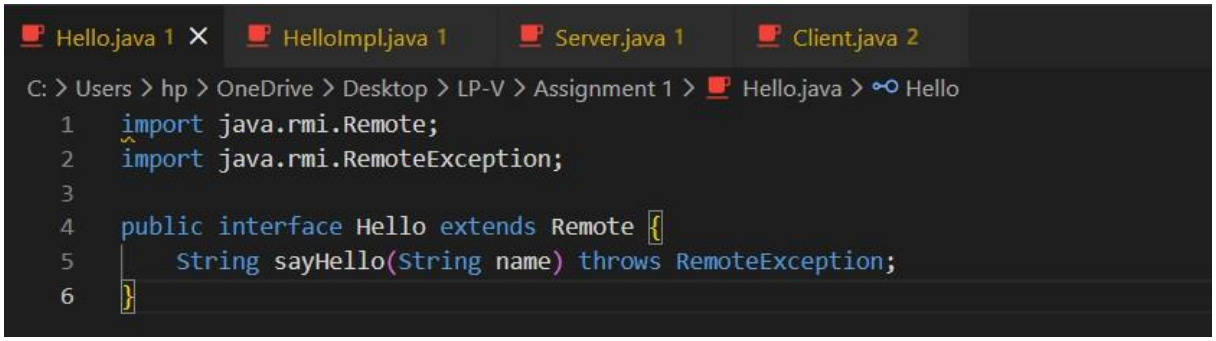
1. **Client** – Requests services from the server
2. **Server** – Provides remote methods
3. **RMI Registry** – Acts as a directory service where remote objects are registered

Working of RMI :-

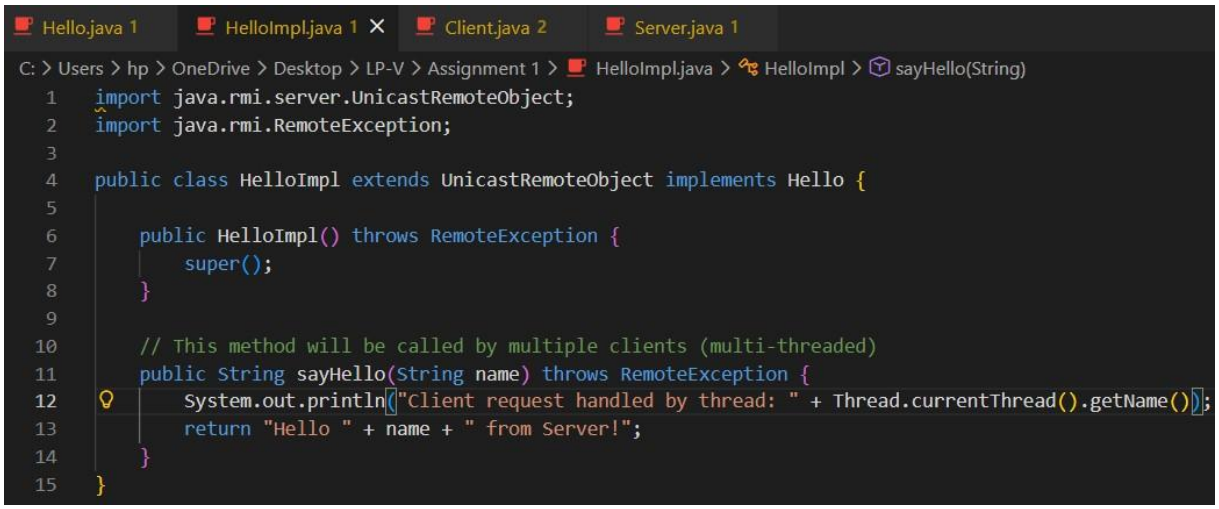
1. The server defines a remote interface that declares methods.
2. The server implements this interface and creates a remote object.
3. The remote object is registered with the RMI registry.
4. The client searches (lookup) the remote object using its name.
5. The client invokes methods on the remote object.
6. The RMI system:
    - Sends the request over the network
    - Executes the method on the server
    - Returns the result back to the client

IMPLEMENTATION :

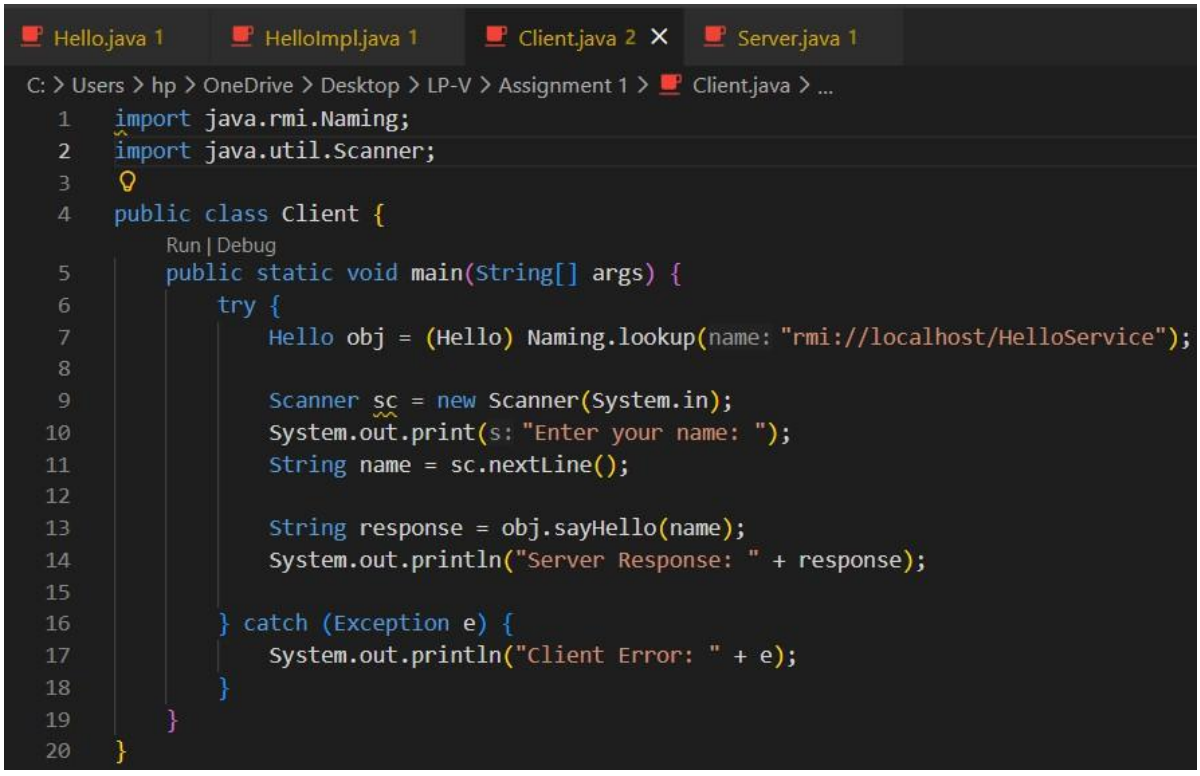
Hello.java



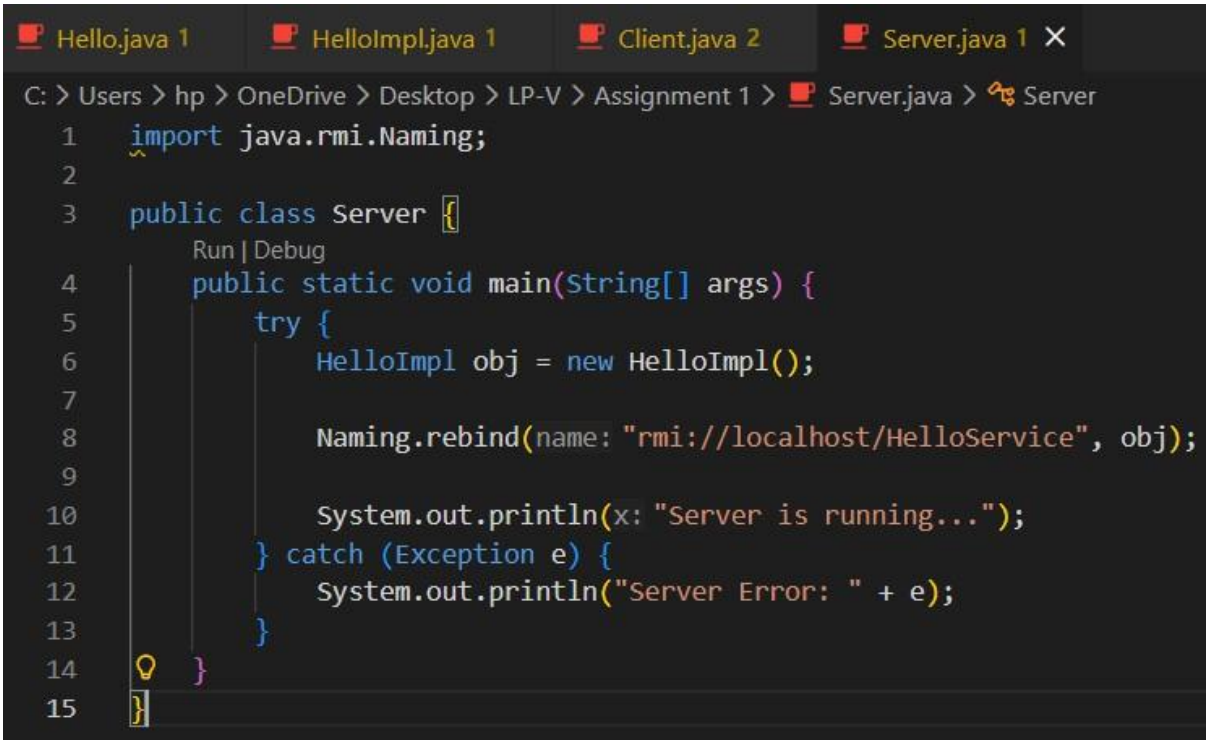
HelloImpl.java



Client.java



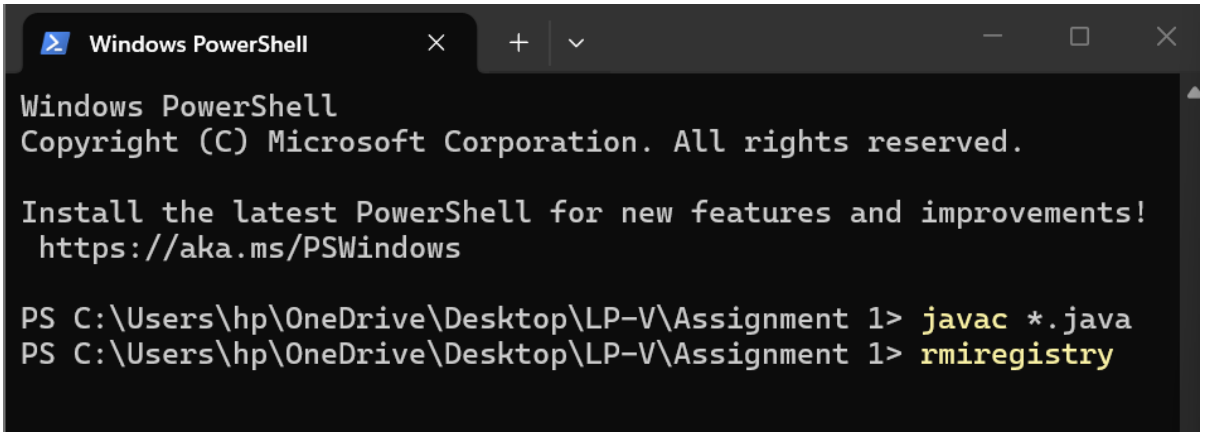
Server.java

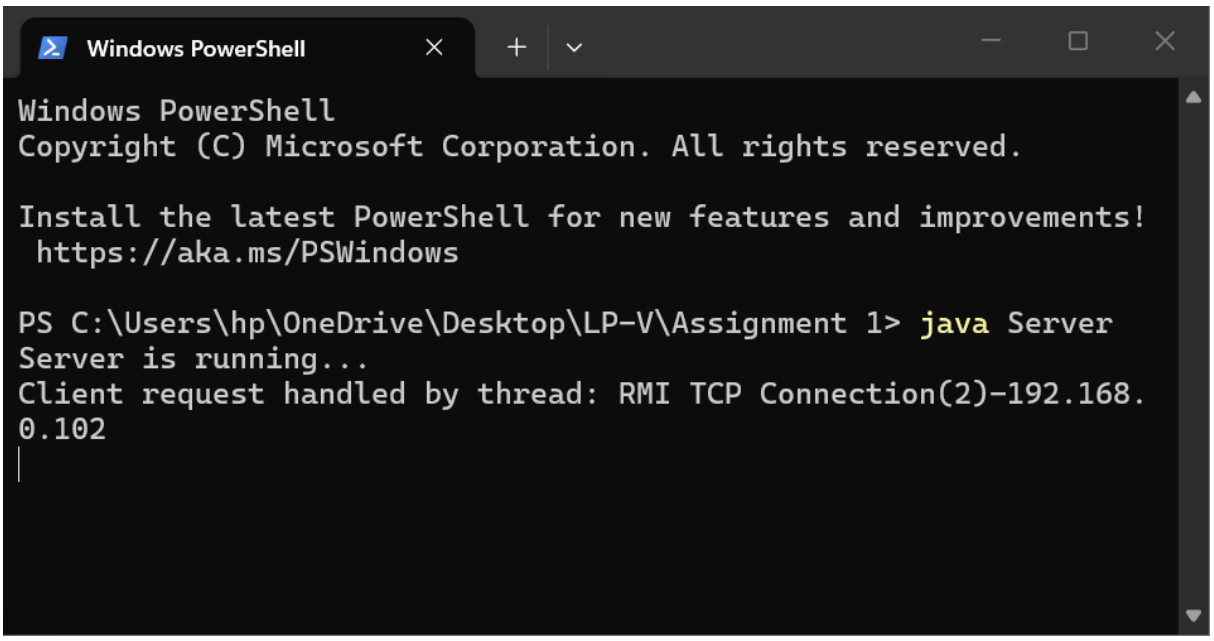


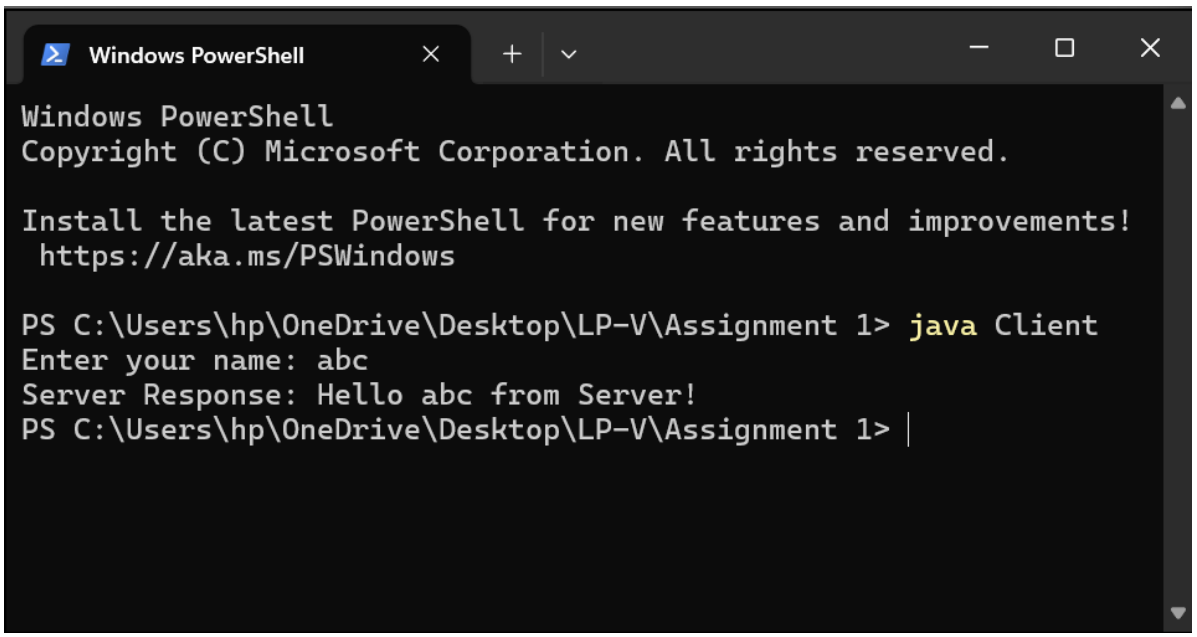
CONLCUSION :

Successfully implemented a multi-threaded client-server communication using Java RMI, where multiple clients interact with the server concurrently.

OUTPUT :


Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.
Install the latest PowerShell for new features and improvements!
https://aka.ms/PSWindows
PS C:\Users\hp\OneDrive\Desktop\LP-V\Assignment 1> javac *.java
PS C:\Users\hp\OneDrive\Desktop\LP-V\Assignment 1> rmiregistry


Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.
Install the latest PowerShell for new features and improvements!
https://aka.ms/PSWindows
PS C:\Users\hp\OneDrive\Desktop\LP-V\Assignment 1> java Server
Server is running...
Client request handled by thread: RMI TCP Connection(2)-192.168.
0.102


Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.
Install the latest PowerShell for new features and improvements!
https://aka.ms/PSWindows
PS C:\Users\hp\OneDrive\Desktop\LP-V\Assignment 1> java Client
Enter your name: abc
Server Response: Hello abc from Server!
PS C:\Users\hp\OneDrive\Desktop\LP-V\Assignment 1>

EXPERIMENT-2

TITLE: Distributed Application using CORBA (Calculator)

AIM: To develop a distributed application using **CORBA** to demonstrate **object brokering** using a calculator service.

TOOLS : Java Programming Environment, JDK 1.8

THEORY:

CORBA :-

CORBA (Common Object Request Broker Architecture) is a standard defined by the Object Management Group (OMG) that enables communication between distributed objects in a network, regardless of the programming language, platform, or location. It is used to build distributed systems where different components interact seamlessly.

At the core of CORBA is the Object Request Broker (ORB), which acts as a middleware that manages communication between the client and server. The ORB allows a client to invoke methods on a remote object without needing to know where the object is located or how it is implemented.

The IDL Programming Model:

The IDL programming model, known as Java IDL, consists of both the Java CORBA ORB and the compiler that maps the IDL to Java bindings that use the Java CORBA ORB, as well as a set of APIs, which can be explored by selecting the prefix from the Packagesection of the API index.

To use the IDL programming model, define remote interfaces using OMG Interface Definition Language (IDL), then compile the interfaces using compiler. When you run the compiler over your interface definition file, it generates the Java version of theinterface, as well as the class code files for the stubs and skeletons that enable applications to hook into the ORB.

Portable Object Adapter (POA) : An object adapter is the mechanism that connects a request using an object reference with the proper code to service that request. The Portable Object Adapter, or POA, is a particular type of object adapter that is defined by the CORBA specification. The POA is designed to meet the following goals:

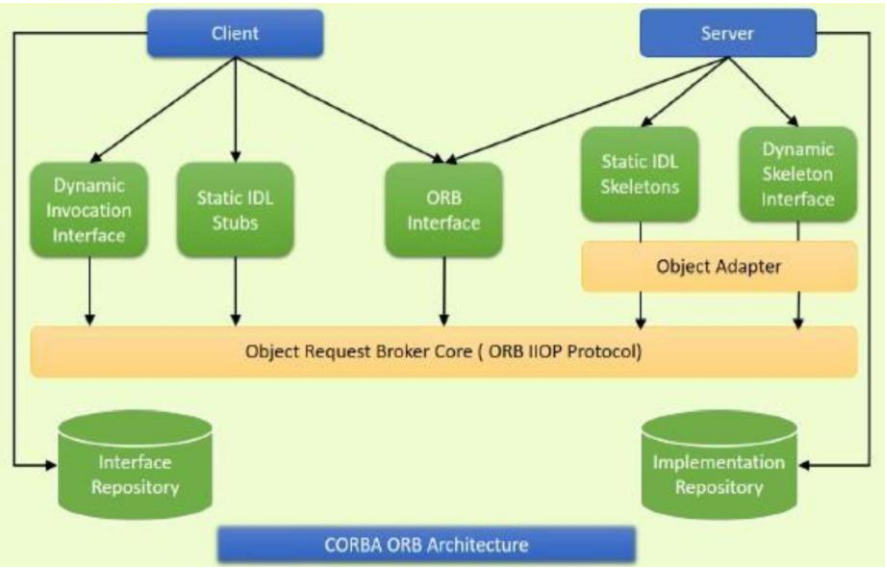
Allow programmers to construct object implementations that are portable between different ORB products.

Key Components of CORBA

1. IDL (Interface Definition Language) - Used to define the interface of remote objects.
2. ORB (Object Request Broker) -Handles communication between client and server.
3. Stub and Skeleton
    - Stub (Client-side): Acts as a proxy for the remote object.
    - Skeleton (Server-side): Receives requests and invokes actual methods.
4. Naming Service - Helps clients locate remote objects using names.

Working of CORBA :-

1. The server defines an interface using IDL.
2. The IDL compiler generates stub and skeleton code.
3. The server implements the methods defined in the IDL.
4. The server registers the object with the naming service.
5. The client looks up the object using its name.
6. The client invokes methods on the remote object.
7. The ORB handles communication and returns the result.



IMPLEMENTATION :

Calculator.idl

C: > Users > hp > OneDrive > Desktop > LP-V > Assignment 2 >

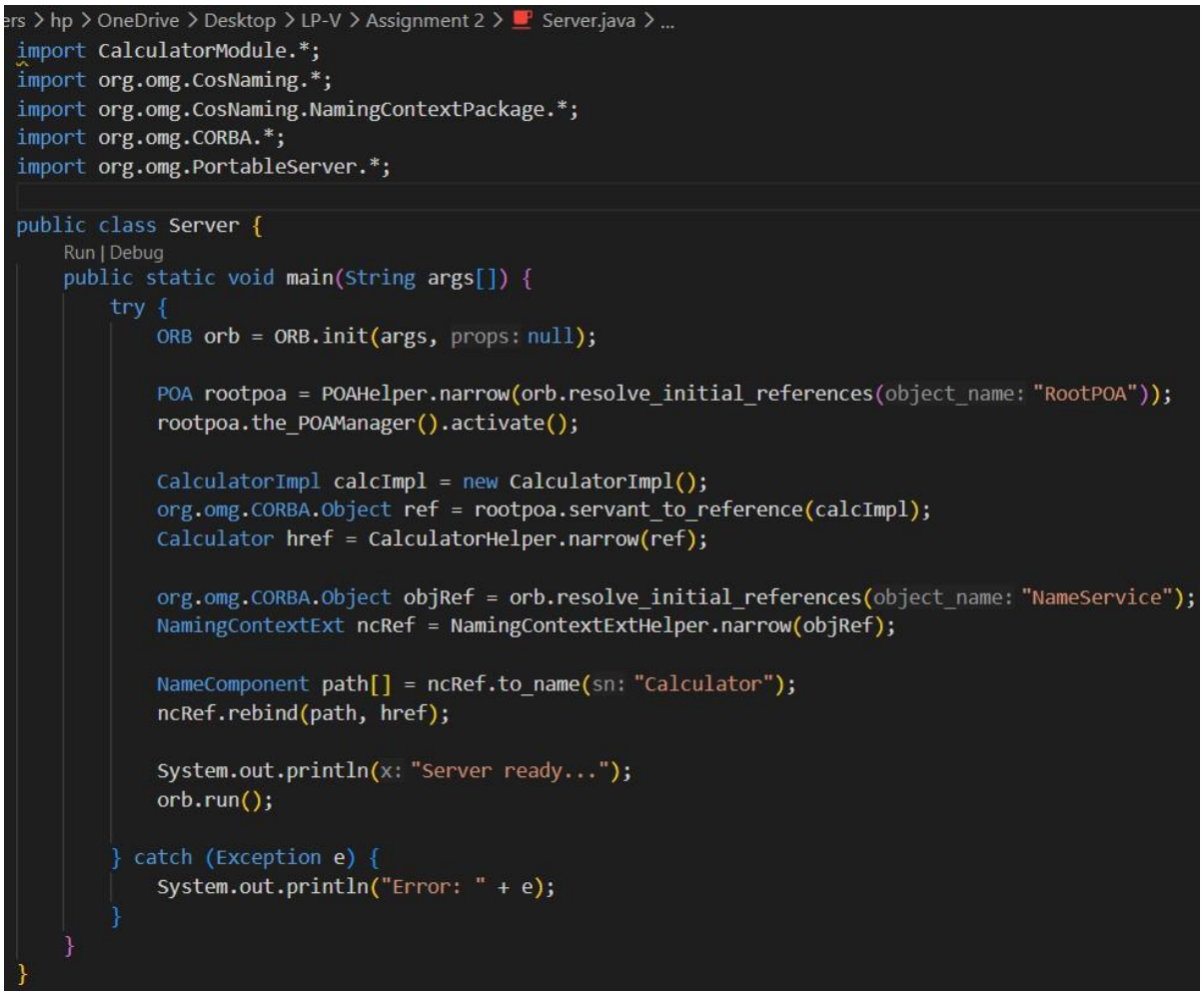
```
module CalculatorModule {
    interface Calculator {
        long add(in long a, in long b);
        long sub(in long a, in long b);
        long mul(in long a, in long b);
        float div(in long a, in long b);
    };
};
```

CalculatorImpl.java

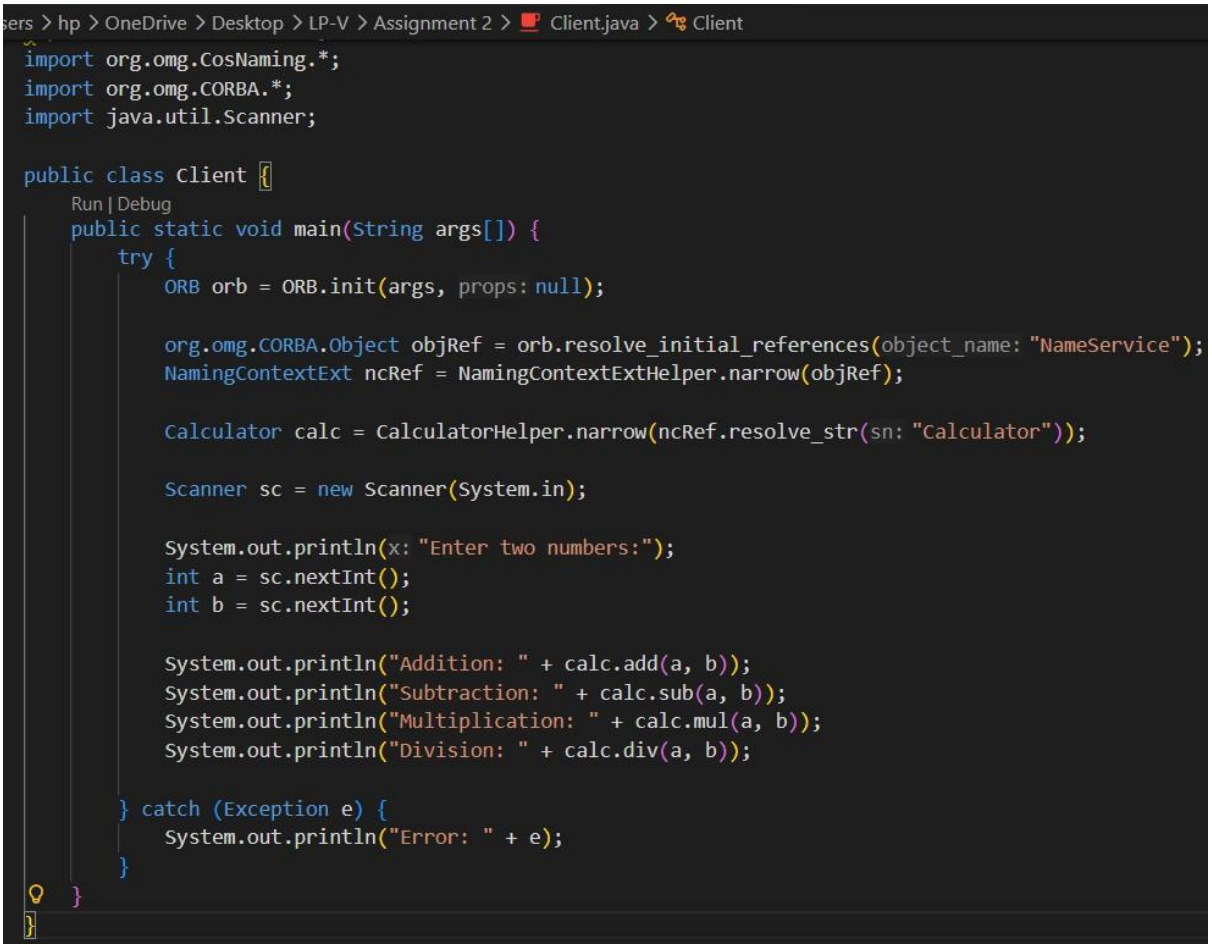
C: > Users > hp > OneDrive > Desktop > LP-V > Assignment 2 > Calculato

```
import CalculatorModule.*;
import org.omg.CORBA.*;

public class CalculatorImpl extends CalculatorPOA {

    public int add(int a, int b) {
        return a + b;
    }

    public int sub(int a, int b) {
        return a - b;
    }

    public int mul(int a, int b) {
        return a * b;
    }

    public float div(int a, int b) {
        if(b == 0) return 0;
        return (float)a / b;
    }
}
```

Server.java


ers > hp > OneDrive > Desktop > LP-V > Assignment 2 > Server.java > ...
import CalculatorModule.*;
import org.omg.CosNaming.*;
import org.omg.CosNaming.NamingContextPackage.*;
import org.omg.CORBA.*;
import org.omg.PortableServer.*;
public class Server {
Run | Debug
public static void main(String args[]) {
try {
ORB orb = ORB.init(args, props: null);
POA rootpoa = POAHelper.narrow(orb.resolve_initial_references(object_name: "RootPOA"));
rootpoa.the_POAManager().activate();
CalculatorImpl calcImpl = new CalculatorImpl();
org.omg.CORBA.Object ref = rootpoa.servant_to_reference(calcImpl);
Calculator href = CalculatorHelper.narrow(ref);
org.omg.CORBA.Object objRef = orb.resolve_initial_references(object_name: "NameService");
NamingContextExt ncRef = NamingContextExtHelper.narrow(objRef);
NameComponent path[] = ncRef.to_name(sn: "Calculator");
ncRef.rebind(path, href);
System.out.println(x: "Server ready...");
orb.run();
} catch (Exception e) {
System.out.println("Error: " + e);
}
}
}

Client.java



```
sers > hp > OneDrive > Desktop > LP-V > Assignment 2 > Client.java > Client
import org.omg.CosNaming.*;
import org.omg.CORBA.*;
import java.util.Scanner;

public class Client {
    Run | Debug
    public static void main(String args[]) {
        try {
            ORB orb = ORB.init(args, props: null);

            org.omg.CORBA.Object objRef = orb.resolve_initial_references(object_name: "NameService");
            NamingContextExt ncRef = NamingContextExtHelper.narrow(objRef);

            Calculator calc = CalculatorHelper.narrow(ncRef.resolve_str(sn: "Calculator"));

            Scanner sc = new Scanner(System.in);

            System.out.println(x: "Enter two numbers:");
            int a = sc.nextInt();
            int b = sc.nextInt();

            System.out.println("Addition: " + calc.add(a, b));
            System.out.println("Subtraction: " + calc.sub(a, b));
            System.out.println("Multiplication: " + calc.mul(a, b));
            System.out.println("Division: " + calc.div(a, b));

        } catch (Exception e) {
            System.out.println("Error: " + e);
        }
    }
}
```

CONLCUSION :

The distributed calculator application was successfully implemented using CORBA.

OUTPUT :

```
C:\Users\hp\OneDrive\Desktop\LP-V\Assignment 2>idlj -fall Calculator.idl

C:\Users\hp\OneDrive\Desktop\LP-V\Assignment 2>javac *.java CalculatorModule/*.java
Note: CalculatorModule\CalculatorPOA.java uses unchecked or unsafe operations.
Note: Recompile with -Xlint:unchecked for details.

C:\Users\hp\OneDrive\Desktop\LP-V\Assignment 2>tnameserv -ORBInitialPort 1050
Initial Naming Context:
IOR:000000000000002b49444c3a6f6d672e6f72672f436f734e616d696e672f4e616d696e67436f6e74
00102000000000e3139322e3136382e302e31303200041a00000045afabcb0000000020000f42400000
0d544e616d65536572766963650000000000000008000000010000001140000000000000020000000100
1010900000001000101000000002600000020002
TransientNameServer: setting port for initial object references to: 1050
Ready.
```

```
Command Prompt - java Ser
Microsoft Windows [Version 10.0.26200.8117]
(c) Microsoft Corporation. All rights reserved.

C:\Users\hp>cd..

C:\Users>cd..

C:\>cd Users\hp\OneDrive\Desktop\LP-V\Assignment 2

C:\Users\hp\OneDrive\Desktop\LP-V\Assignment 2>java Server -ORBI
nitialPort 1050
Server ready...
```

```
Command Prompt
C:\Users>cd..

C:\>cd Users\hp\OneDrive\Desktop\LP-V\Assignment 2

C:\Users\hp\OneDrive\Desktop\LP-V\Assignment 2>java Client -ORBI
nitialPort 1050
Enter two numbers:
12 24
Addition: 36
Subtraction: -12
Multiplication: 288
Division: 0.5

C:\Users\hp\OneDrive\Desktop\LP-V\Assignment 2>
```

EXPERIMENT-3

TITLE: Develop a Distributed System to find Sum of N Elements in an Array using MPI/OpenMP

AIM: To compute the sum of N elements of an array by dividing the workload among multiple processors/threads using parallel programming (OpenMP/MPI) and to display intermediate partial sums computed by each processor.

TOOLS: Java Programming Environment, JDK1.8 or higher, MPI Library (mpi.jar), MPJ Express (mpj.jar)

THEORY:

Introduction to Distributed Computing

Distributed computing is a model in which a problem is divided into smaller sub-tasks and executed simultaneously on multiple processors or computing nodes. Each processor performs a part of the computation and results are combined to obtain the final output.

This approach improves:

- Execution speed
- Efficiency
- Resource utilization

Parallel Computing Model

The problem follows the data parallelism model, where:

- Same operation (addition) is applied
- Different processors work on different data segments

OpenMP Concept (Shared Memory Model) :-

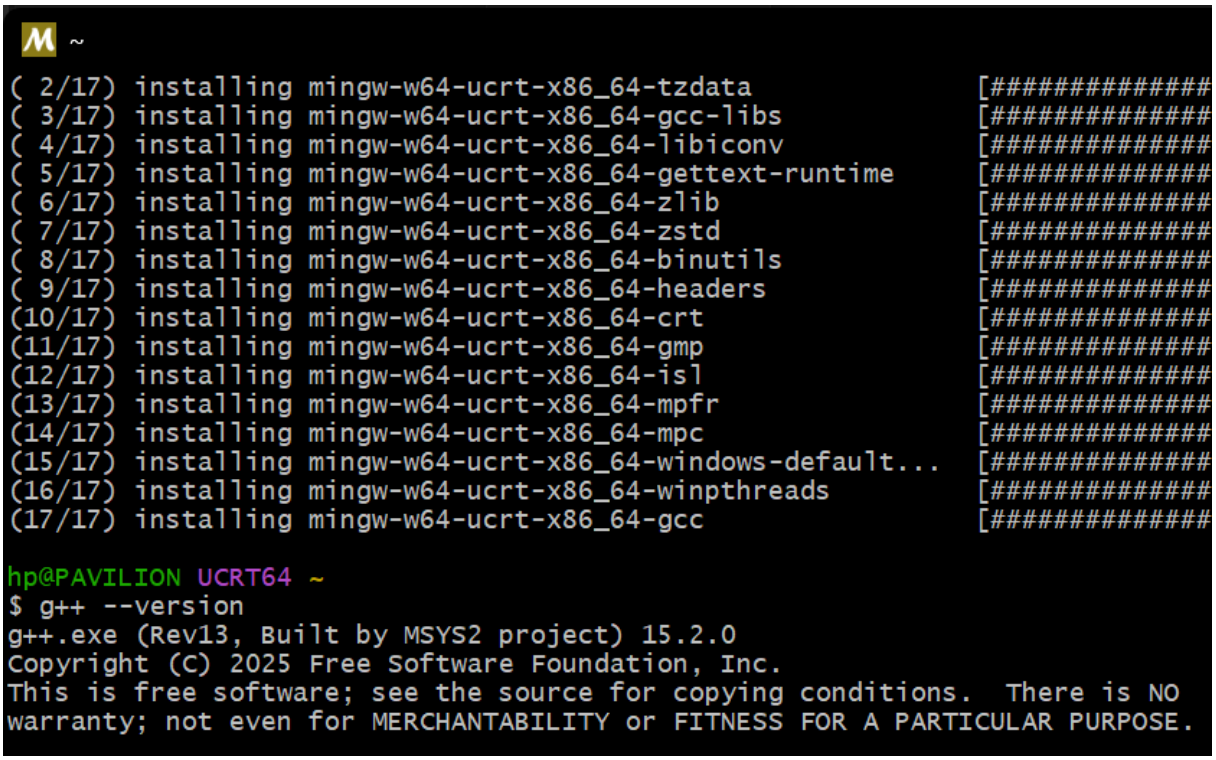
OpenMP is an API used for parallel programming in C/C++.

Key Features:

- Uses threads instead of processes
- Shared memory architecture
- Easy to implement using compiler directives

METHOD USED: OPENMP IMPLEMENTATION

Step 1: Install via MSYS2



Step 2: Compile Program

Open terminal in file location:

g++ -fopenmp distributed_sum.cpp -o sum.exe

Step 3: Run Program

sum.exe

IMPLEMENTATION :

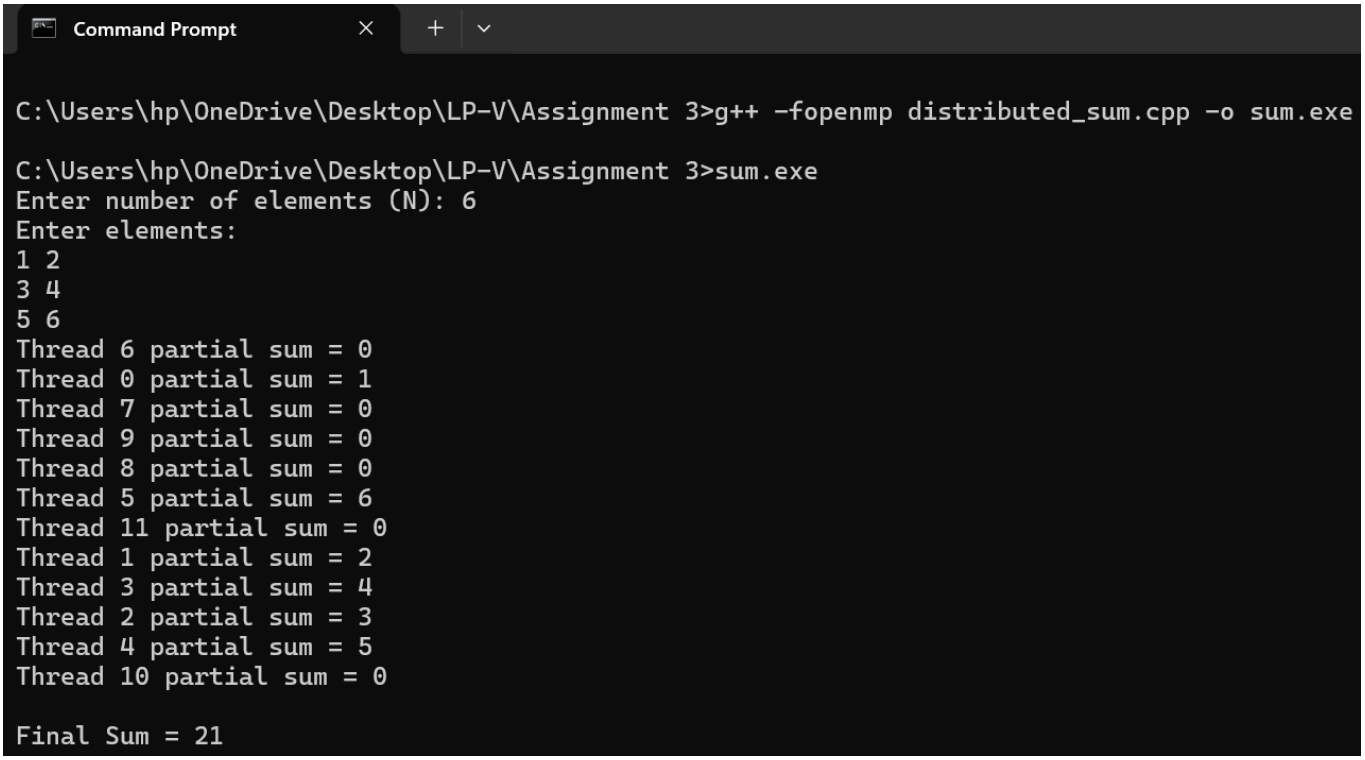
distributed_sum.c++

```cpp
#include <iostream>
#include <vector>
#include <omp.h>
using namespace std;

int main() {
    int N;
    cout << "Enter number of elements (N): ";
    cin >> N;
    vector<int> arr(N);

    cout << "Enter elements:\n";
    for (int i = 0; i < N; i++) {
        cin >> arr[i];
    }

    int totalSum = 0;

    #pragma omp parallel
    {
        int threadSum = 0;
        #pragma omp for
        for (int i = 0; i < N; i++) {
            threadSum += arr[i];
        }
        #pragma omp critical
        {
            cout << "Thread " << omp_get_thread_num()
                 << " partial sum = " << threadSum << endl;

            totalSum += threadSum;
        }
    }
    cout << "\nFinal Sum = " << totalSum << endl;
    return 0;
}
```

CONLCUSION :

The experiment demonstrates how distributed computing techniques like OpenMP can be used to divide computational tasks among multiple processors.

OUTPUT :



EXPERIMENT - 4

TITLE: Berkeley Algorithm for Clock Synchronization

AIM: To implement the Berkeley algorithm for synchronizing clocks in a distributed system.

TOOLS: Python3, Eclipse IDE

THEORY:

Berkeley Algorithm :-

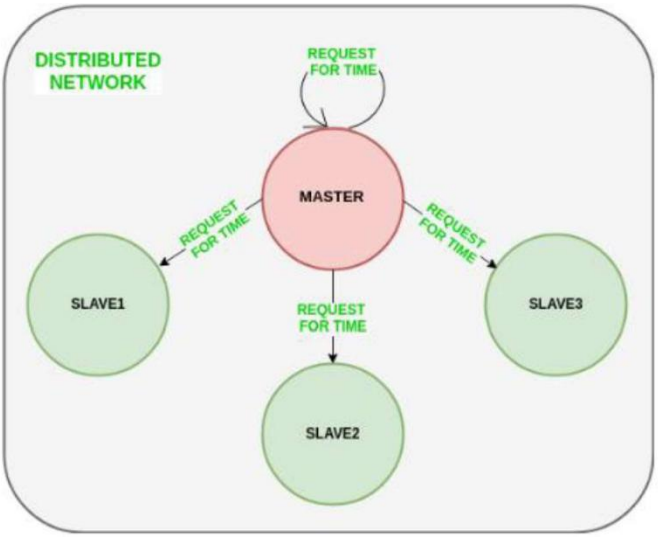
In distributed systems, each computer (node) has its own clock, which may differ due to drift. The Berkeley Algorithm is used to synchronize these clocks without relying on an external time source.

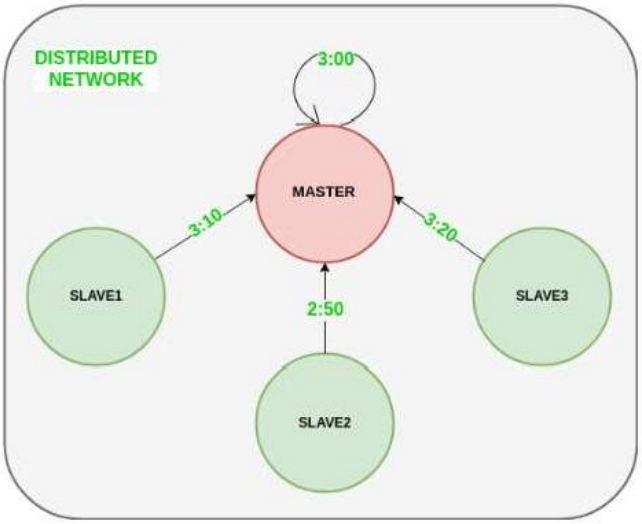
Working Principle:

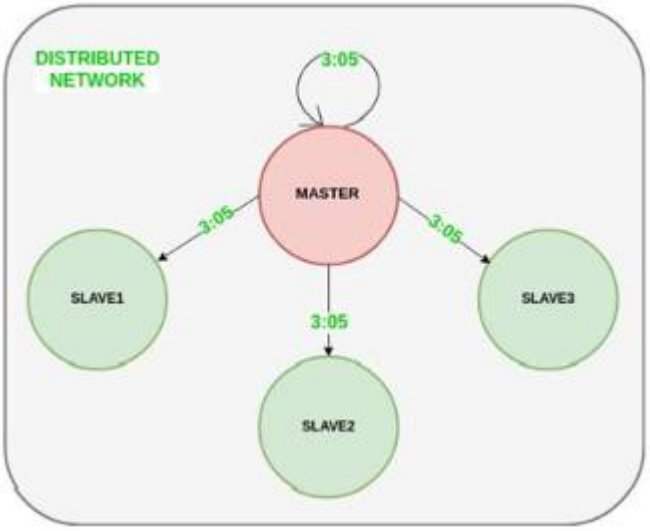
- One node is selected as Master (Time Server)
- Other nodes act as Slaves (Clients)

Steps:

1. Master sends request to all clients asking their local time
2. Clients respond with their current time
3. Master calculates the average time (excluding faulty clocks if needed)
4. Master computes time difference (offset) for each client
5. Master sends adjustment values to clients
6. Clients update their clocks accordingly


DISTRIBUTED
NETWORK
REQUEST
FOR TIME
MASTER
REQUEST
FOR TIME
REQUEST
FOR TIME
SLAVE1
SLAVE3
REQUEST
FOR TIME
SLAVE2


DISTRIBUTED
NETWORK
3:00
MASTER
3:10
3:20
SLAVE1
SLAVE3
2:50
SLAVE2


DISTRIBUTED
NETWORK
3:05
MASTER
3:05
3:05
SLAVE1
SLAVE3
3:05
SLAVE2

IMPLEMENTATION :

Client.py

Client.py Server.py

C: > Users > hp > OneDrive > Desktop > LP-V > Assignment 4 > Clier

```python
# client.py
import socket
import time
import random

HOST = '127.0.0.1'
PORT = 5000

s = socket.socket()
s.connect((HOST, PORT))

# Simulated clock (random offset)
local_time = time.time() + random.randint(-5, 5)

msg = s.recv(1024).decode()

if msg == "TIME":
    print("Sending time:", local_time)
    s.send(str(local_time).encode())

# Receive adjustment
adjustment = float(s.recv(1024).decode())

# Adjust clock
local_time += adjustment

print("Adjusted Time:", local_time)

s.close()
```

Server.java

Client.py Server.py

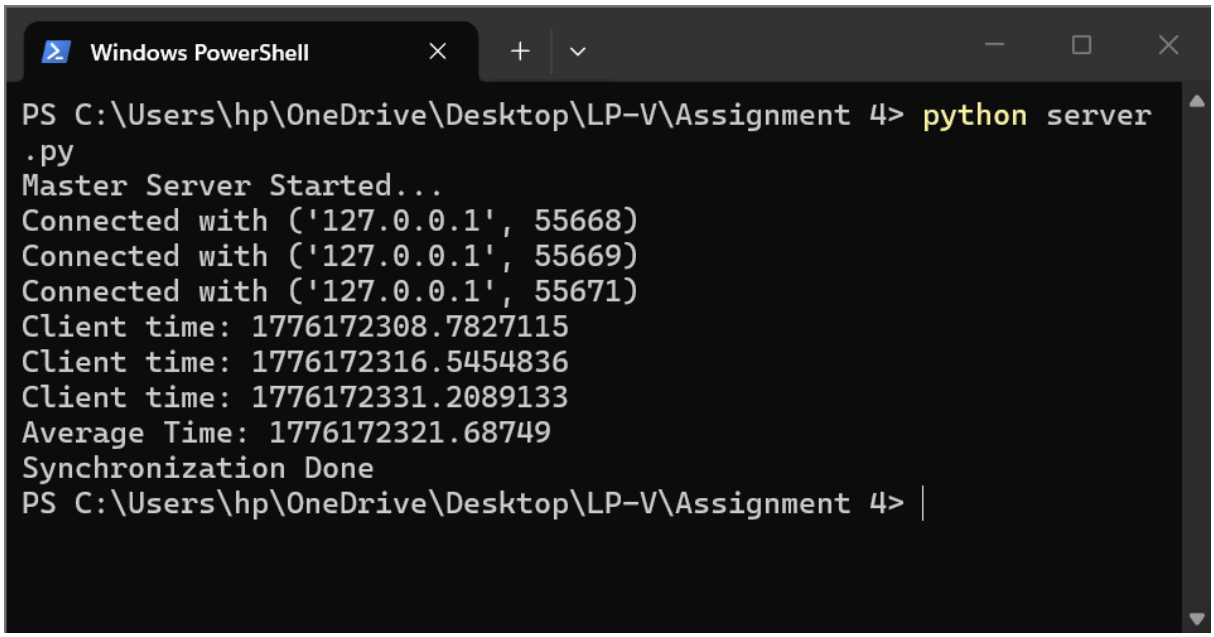
C: > Users > hp > OneDrive > Desktop > LP-V > Assignment 4 > Server.p

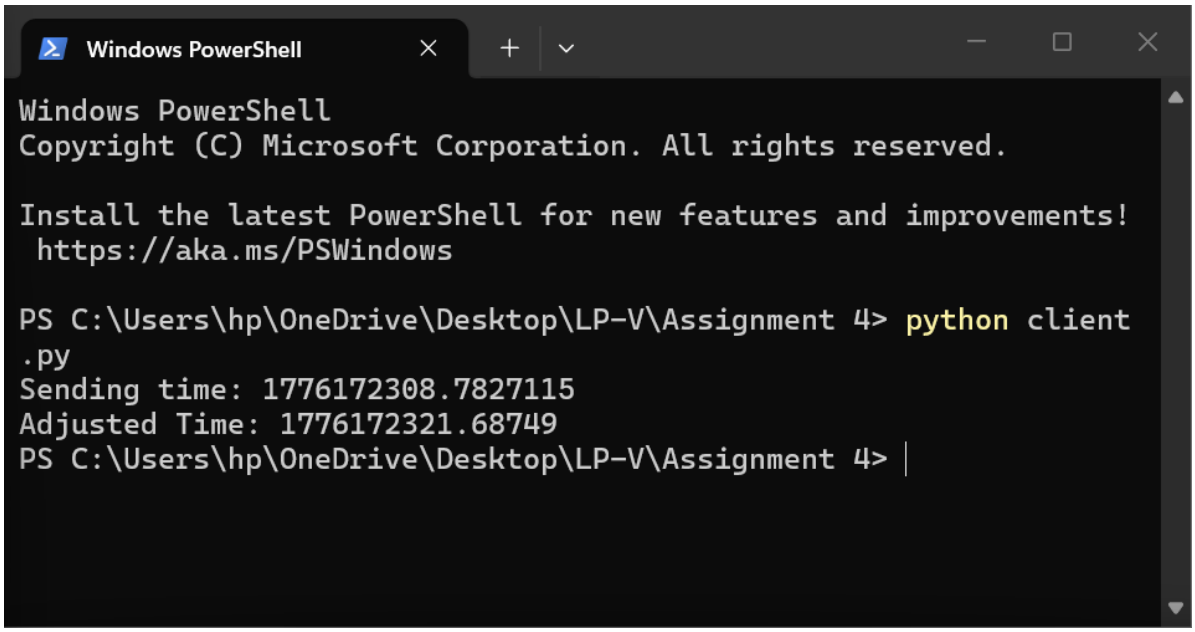
```python
# server.py
import socket
import time
clients = []
times = []
HOST = '127.0.0.1'
PORT = 5000

s = socket.socket()
s.bind((HOST, PORT))
s.listen(5)
print("Master Server Started...")

for i in range(3):
    conn, addr = s.accept()
    print("Connected with", addr)
    clients.append(conn)

for c in clients:
    c.send(b"TIME")

for c in clients:
    client_time = float(c.recv(1024).decode())
    print("Client time:", client_time)
    times.append(client_time)

server_time = time.time()
times.append(server_time)
avg_time = sum(times) / len(times)
print("Average Time:", avg_time)

for i in range(len(clients)):
    adjustment = avg_time - times[i]
    clients[i].send(str(adjustment).encode())
print("Synchronization Done")
s.close()
```

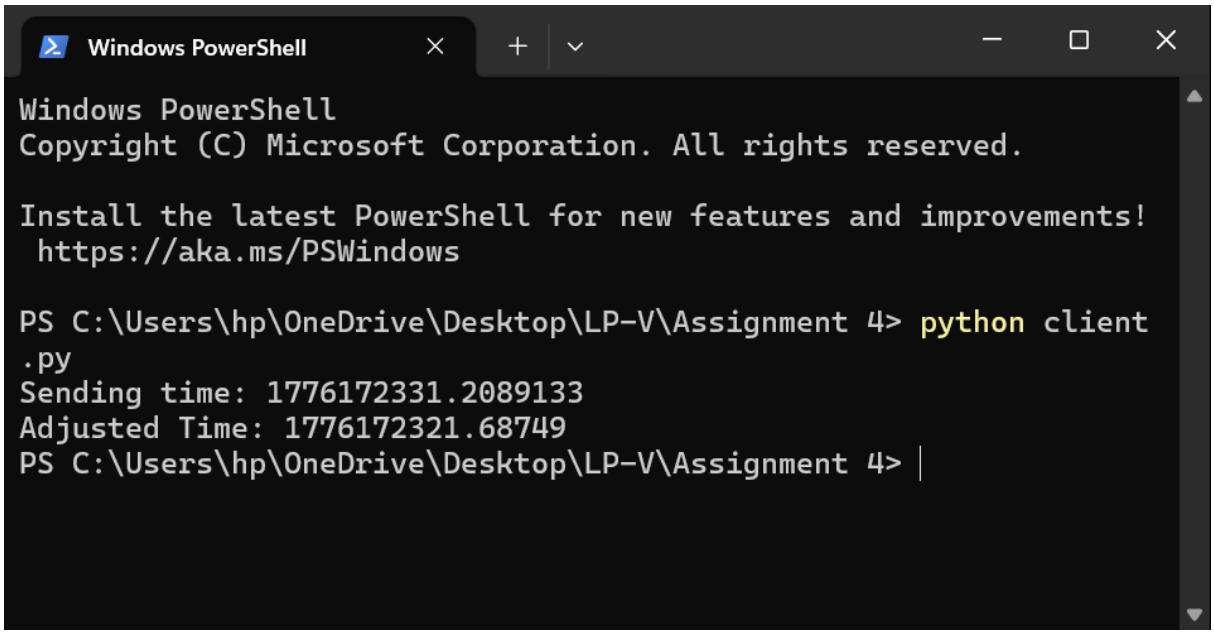
CONLCUSION :

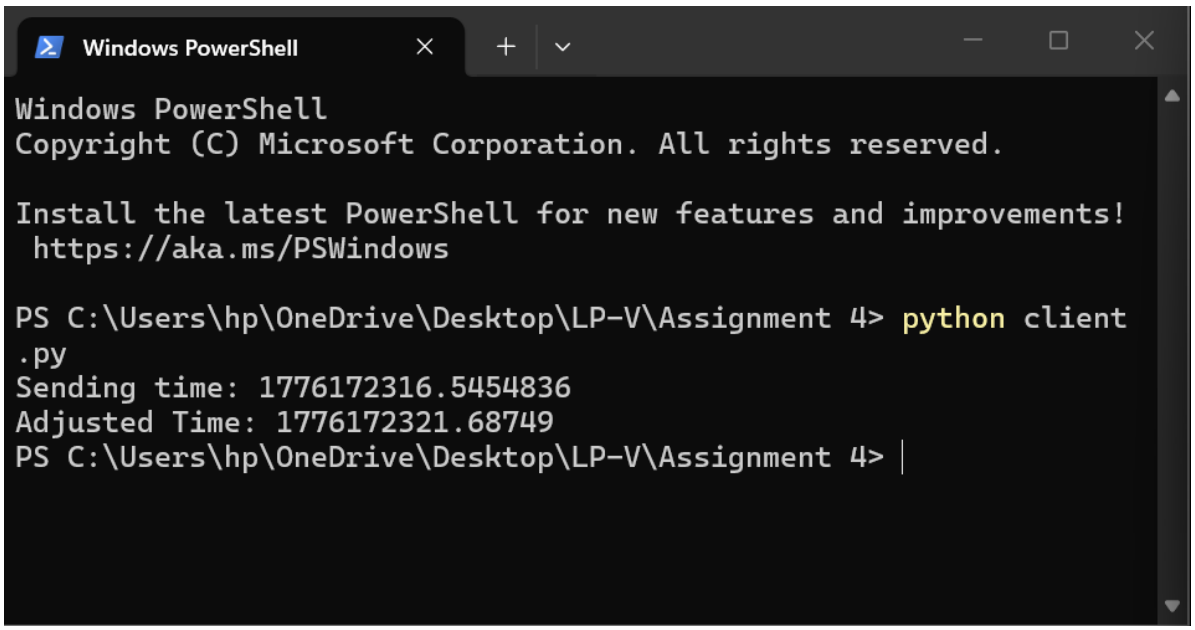
In this way successfully implemented Berkeley algorithm.

OUTPUT : -


Windows PowerShell
PS C:\Users\hp\OneDrive\Desktop\LP-V\Assignment 4> python server
.py
Master Server Started...
Connected with ('127.0.0.1', 55668)
Connected with ('127.0.0.1', 55669)
Connected with ('127.0.0.1', 55671)
Client time: 1776172308.7827115
Client time: 1776172316.5454836
Client time: 1776172331.2089133
Average Time: 1776172321.68749
Synchronization Done
PS C:\Users\hp\OneDrive\Desktop\LP-V\Assignment 4>


Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.
Install the latest PowerShell for new features and improvements!
https://aka.ms/PSWindows
PS C:\Users\hp\OneDrive\Desktop\LP-V\Assignment 4> python client
.py
Sending time: 1776172308.7827115
Adjusted Time: 1776172321.68749
PS C:\Users\hp\OneDrive\Desktop\LP-V\Assignment 4>


Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.
Install the latest PowerShell for new features and improvements!
https://aka.ms/PSWindows
PS C:\Users\hp\OneDrive\Desktop\LP-V\Assignment 4> python client
.py
Sending time: 1776172331.2089133
Adjusted Time: 1776172321.68749
PS C:\Users\hp\OneDrive\Desktop\LP-V\Assignment 4>


Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.
Install the latest PowerShell for new features and improvements!
https://aka.ms/PSWindows
PS C:\Users\hp\OneDrive\Desktop\LP-V\Assignment 4> python client
.py
Sending time: 1776172316.5454836
Adjusted Time: 1776172321.68749
PS C:\Users\hp\OneDrive\Desktop\LP-V\Assignment 4>

EXPERIMENT - 5

TITLE: Token Ring Based Mutual Exclusion

AIM: To implement the Token Ring Mutual Exclusion Algorithm for a distributed system and demonstrate how processes access the critical section using a circulating token.

TOOLS : Java Programming Environment, JDK 1.8

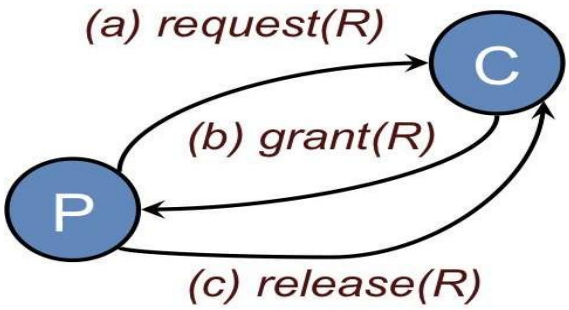
THEORY:

Central server algorithm :-

The central server algorithm simulates a single processor system. One process in the distributed system is elected as the coordinator (Figure 1). When a process wants to enter a resource, it sends a request message (Figure 1a) identifying the resource, if there are more than one, to the coordinator.

If nobody is currently in the section, the coordinator sends back a grant message (Figure 1b) and marks that process as using the resource. If, however, another process has previously claimed the resource, the server simply does not reply, so the requesting process is blocked. The coordinator keeps state on which process is currently granted the resource and which processes are requesting the resource. The list of requestors is a first-come, first-served queue per resource. If some process has been granted access to the resource but has not released it, any incoming grant requests are queued on the coordinator. When a coordinator receives a release(R) message, it sends a grant message to the next process in the queue for resource R.

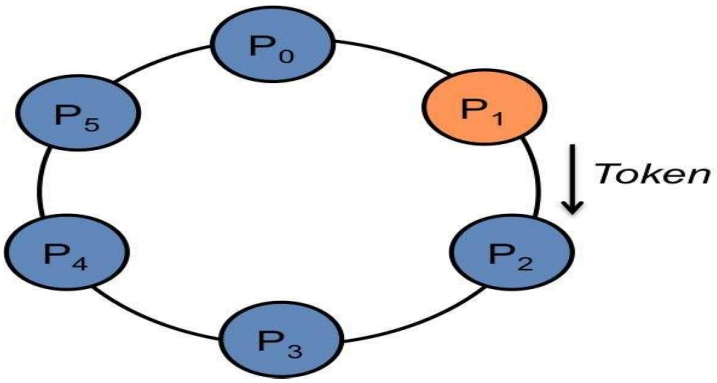
When a process is done with its resource, it sends a release message (Figure 1c) to the coordinator. The coordinator then can send a grant message to the next process in its queue of processes requesting a resource (if any).



Token Ring Algorithm :-

he Token Ring Algorithm is a distributed mutual exclusion algorithm where:

- All processes are arranged in a logical ring structure.
- A special message called a token circulates in the ring.
- Only the process holding the token can enter the critical section (CS).
- After finishing execution, the process passes the token to the next process.



Working :-

1. Token is initially given to one process.
2. Token moves in sequence: P1 → P2 → P3 → ... → Pn → P1
3. If a process wants to enter CS:
    - Wait for token
    - Execute CS
    - Pass token

Algorithm :-

1. Initialize N processes in a ring.
2. Assign token to process 0.
3. For each process:

- If it has the token:
    - Enter critical section
    - Execute task
    - Pass token to next process

4. Repeat for multiple cycles.

IMPLEMENTATION :

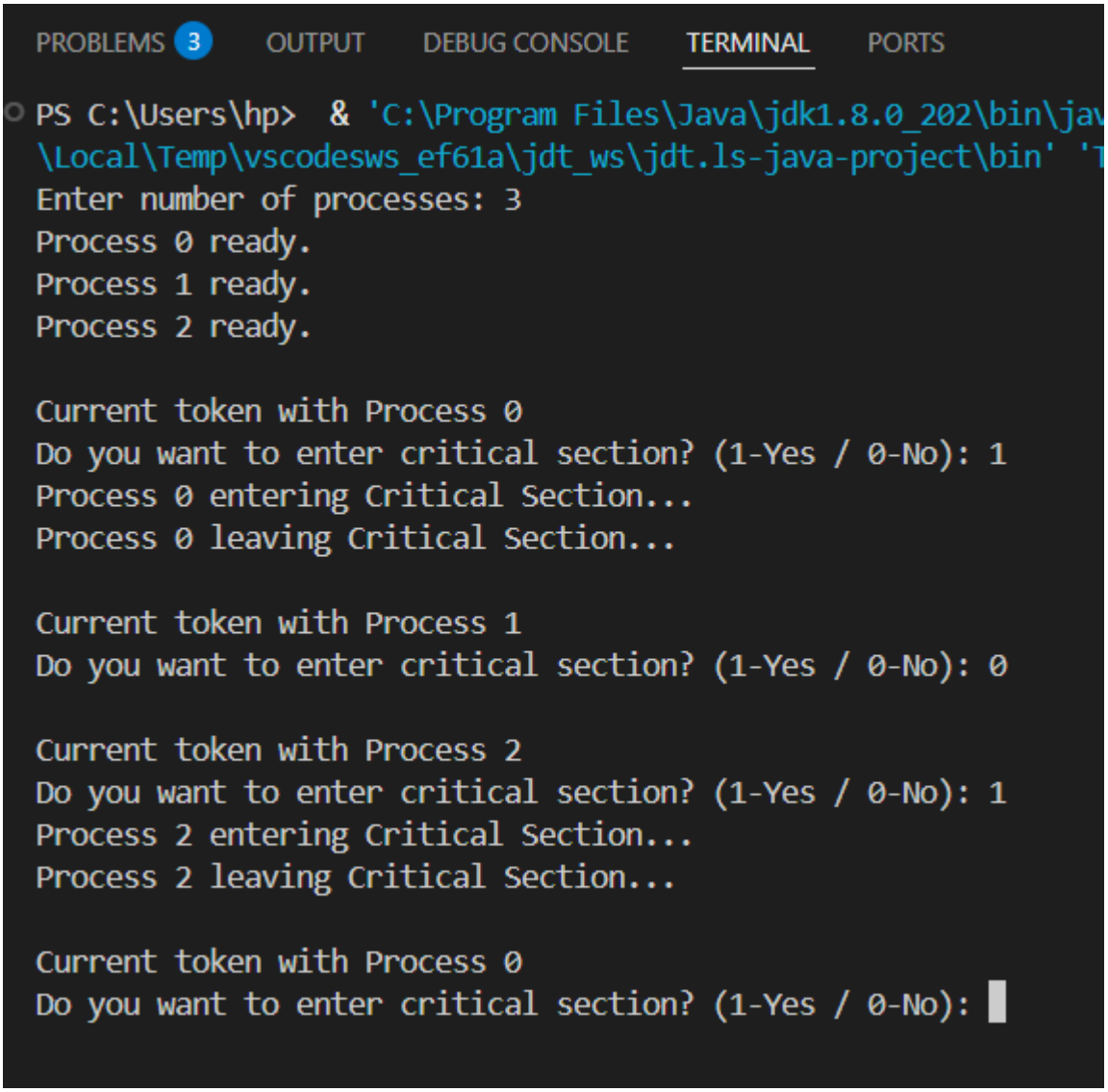
TokenRing.java

```java
import java.util.Scanner;
class TokenRing {
    int n;
    int token;
    TokenRing(int n) {
        this.n = n;
        this.token = 0;
    }
    void runSimulation() {
        Scanner sc = new Scanner(System.in);
        for (int i = 0; i < n; i++) {
            System.out.println("Process " + i + " ready.");
        }
        int choice;
        do {
            System.out.println("\nCurrent token with Process " + token);
            System.out.print(s: "Do you want to enter critical section? (1-Yes / 0-No): ");
            choice = sc.nextInt();
            if (choice == 1) {
                System.out.println("Process " + token + " entering Critical Section...");
                try {
                    Thread.sleep(millis: 1000);
                } catch (InterruptedException e) {}
                System.out.println("Process " + token + " leaving Critical Section...");
            }
            token = (token + 1) % n;
        } while (true);
    }
    Run | Debug
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print(s: "Enter number of processes: ");
        int n = sc.nextInt();
        TokenRing ring = new TokenRing(n);
        ring.runSimulation();
    } }
```

CONLCUSION :

In this way successfully implemented Token Ring algorithm and understood its working.

OUTPUT : -


PROBLEMS 3
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
PS C:\Users\hp> & 'C:\Program Files\Java\jdk1.8.0_202\bin\jav
\Local\Temp\vscodesws_ef61a\jdt_ws\jdt.ls-java-project\bin' '
Enter number of processes: 3
Process 0 ready.
Process 1 ready.
Process 2 ready.
Current token with Process 0
Do you want to enter critical section? (1-Yes / 0-No): 1
Process 0 entering Critical Section...
Process 0 leaving Critical Section...
Current token with Process 1
Do you want to enter critical section? (1-Yes / 0-No): 0
Current token with Process 2
Do you want to enter critical section? (1-Yes / 0-No): 1
Process 2 entering Critical Section...
Process 2 leaving Critical Section...
Current token with Process 0
Do you want to enter critical section? (1-Yes / 0-No):

EXPERIMENT - 6

TITLE: Leader Election using Bully and Ring Algorithms

AIM: To implement Bully Algorithm and Ring Algorithm for leader election in a distributed system.

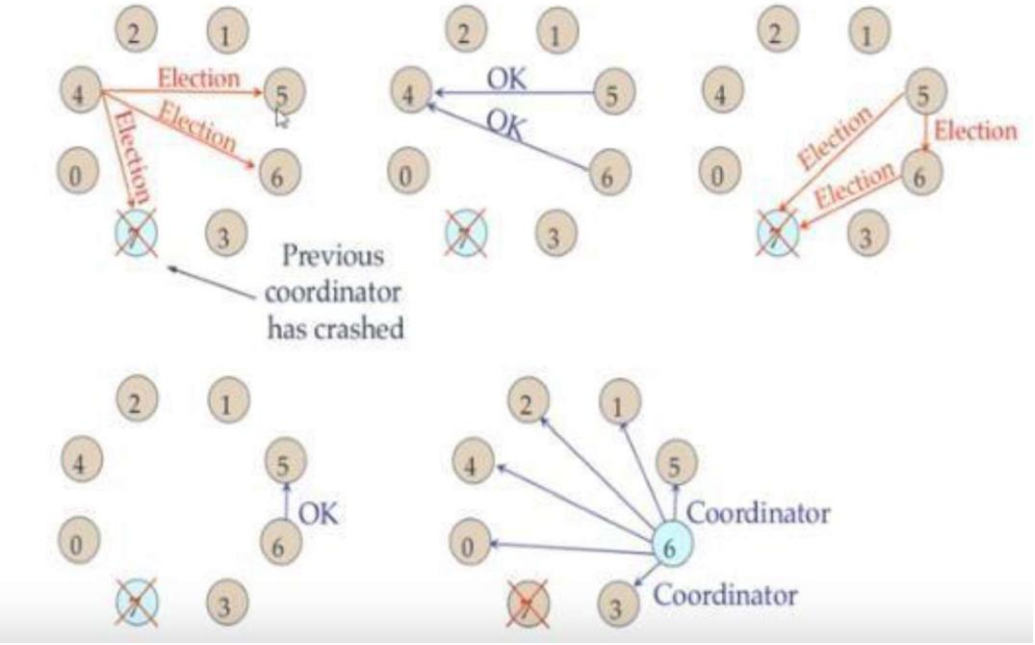
TOOLS : Java Programming Environment, JDK 1.8

THEORY:

Leader Election Problem

In distributed systems, when a coordinator (leader) fails, a new leader must be selected among processes.

1. **Bully Algorithm**



The Bully Algorithm is a leader election algorithm used in distributed systems to select a coordinator (leader) among multiple processes. It was proposed by Hector Garcia-Molina.

In a distributed system, each process is assigned a unique ID, and the process with the highest ID is chosen as the leader**.** The algorithm is called “bully” because the process with the highest ID always dominates (or “bullies”) the others to become the coordinator.

Working Principle

When a process detects that the current coordinator has failed, it initiates an election process:

- The process sends an ELECTION message to all processes with higher IDs.
- If no process responds, it becomes the leader.
- If any higher-ID process responds with an OK message, that process takes over the election.
- This process continues until the highest active process becomes the leader.
- The new leader sends a COORDINATOR message to all other processes announcing itself as the leader.

2. Ring Algorithm

The Ring Algorithm is a leader election algorithm used in distributed systems where processes are arranged in a logical ring structure**.** In this algorithm, each process communicates only with its next neighbor in the ring.
The main goal of the algorithm is to elect a coordinator (leader), and the process with the highest ID is selected as the leader**.**

Working Principle

When a process detects that the current coordinator has failed, it starts an election process:

- The initiator process creates an ELECTION message containing its ID and sends it to the next active process in the ring.
- Each process that receives the message:
    1. Adds its own ID to the message.
    2. Forwards the message to the next process.
- This continues until the message comes back to the initiator.
- The initiator then checks all collected IDs and identifies the highest ID.
- The process with the highest ID is selected as the leader.
- A COORDINATOR message is then passed around the ring to inform all processes about the new leader.

IMPLEMENTATION :

BullyAlgorithm.java

```java
import java.util.Scanner;
public class BullyAlgorithm {
    static int n;
    static int[] processes;
    static boolean[] active;
    Run | Debug
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print(s: "Enter number of processes: ");
        n = sc.nextInt();
        processes = new int[n];
        active = new boolean[n];

        for (int i = 0; i < n; i++) {
            processes[i] = i + 1;
            active[i] = true;
        }

        System.out.print(s: "Enter process to crash: ");
        int crash = sc.nextInt();
        active[crash - 1] = false;
        System.out.print(s: "Enter process initiating election: ");
        int init = sc.nextInt();
        bullyElection(init - 1);
    }
    static void bullyElection(int init) {
        int leader = init;

        for (int i = init + 1; i < n; i++) {
            if (active[i]) {
                System.out.println("Process " + processes[init] +
                        " sends ELECTION to Process " + processes[i]);
                leader = i;
            }
        }
```

```
        if (leader != init) {
            bullyElection(leader);
        } else {
            System.out.println("Process " + processes[leader] + " becomes LEADER");
        }
    }
}
```

RingAlgorithm.java

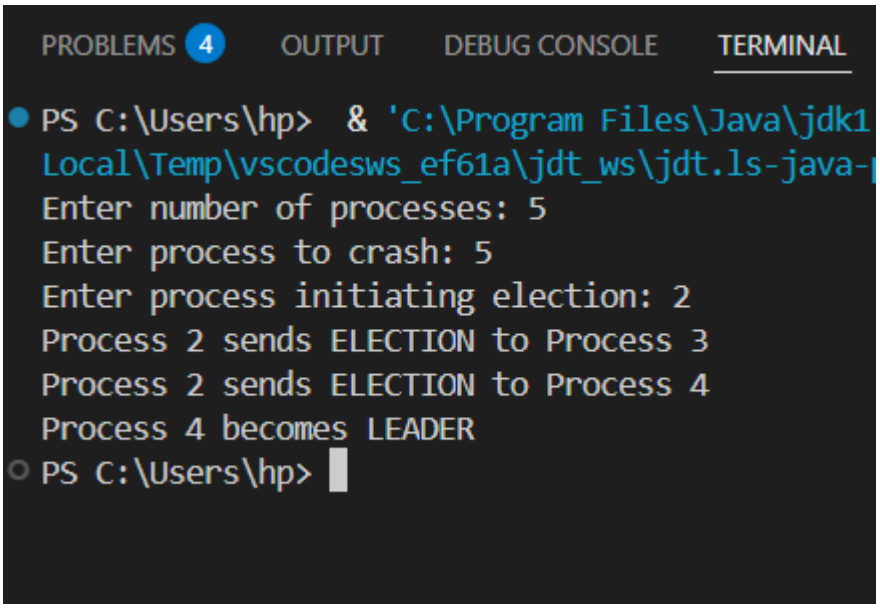
```
import java.util.*;
public class RingAlgorithm {
    Run | Debug
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print(s: "Enter number of processes: ");
        int n = sc.nextInt();
        int[] processes = new int[n];
        boolean[] active = new boolean[n];

        for (int i = 0; i < n; i++) {
            processes[i] = i + 1;
            active[i] = true;
        }

        System.out.print(s: "Enter process to crash: ");
        int crash = sc.nextInt();
        active[crash - 1] = false;
        System.out.print(s: "Enter initiator process: ");
        int init = sc.nextInt() - 1;
        List<Integer> election = new ArrayList<>();
        int i = init;

        do {
            if (active[i]) {
                election.add(processes[i]);
                System.out.println("Process " + processes[i] + " passes message");
            }
            i = (i + 1) % n;
        } while (i != init);
        int leader = Collections.max(election);
        System.out.println("Process " + leader + " becomes LEADER");
    }
}
```

CONLCUSION :

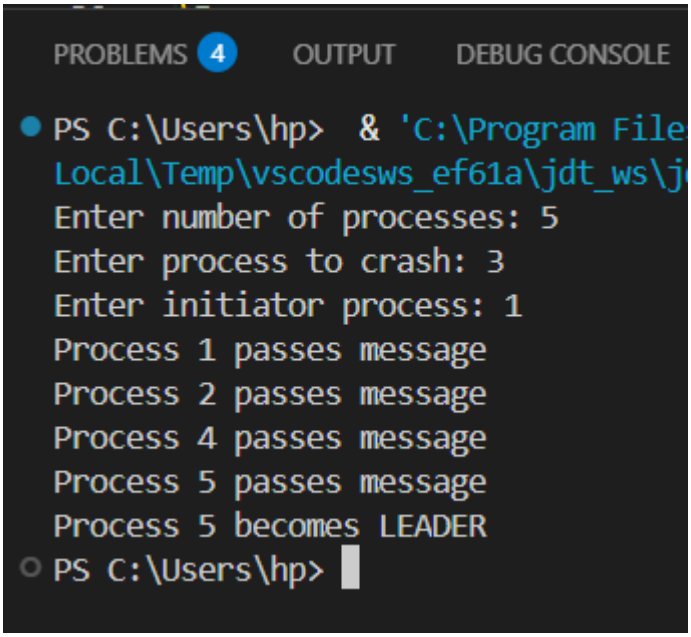
In this way successfully implemented Bully and Ring algorithm and understood its working.

OUTPUT : -

Bully Algorithm



Ring Algorithm



EXPERIMENT - 7

TITLE: Create a simple web service and write any distributed application to consume the web service.

AIM: To create a simple web service that performs addition of two numbers and write a client application to consume it.

TOOLS: JDK 8 or above, Web Browser – Google Chrome, Command Prompt / Terminal, SOAP Testing Tool (Optional) – SoapUI

THEORY:

Web Service :-

A Web Service is a software system that enables communication between two applications over a network using standard protocols such as HTTP. It allows different applications (written in different languages) to interact with each other.

SOAP :-

SOAP (Simple Object Access Protocol) is a messaging protocol used for exchanging structured information in web services.

Key features:

1. Uses XML format for messages
2. Works over HTTP/HTTPS
3. Platform and language independent
4. More secure and strict than REST
5. Ring Algorithm

JAX-WS :-

JAX-WS (Java API for XML Web Services) is a Java technology used to create SOAP-based web services.

It provides:

- Easy creation of web services in Java
- Annotation-based programming model
- Automatic generation of WSDL (Web Service Description Language)

Key Components of JAX-WS

1. Web Service Provider (Server) - Provides services to clients

2. Web Service Consumer (Client) - Requests services from server

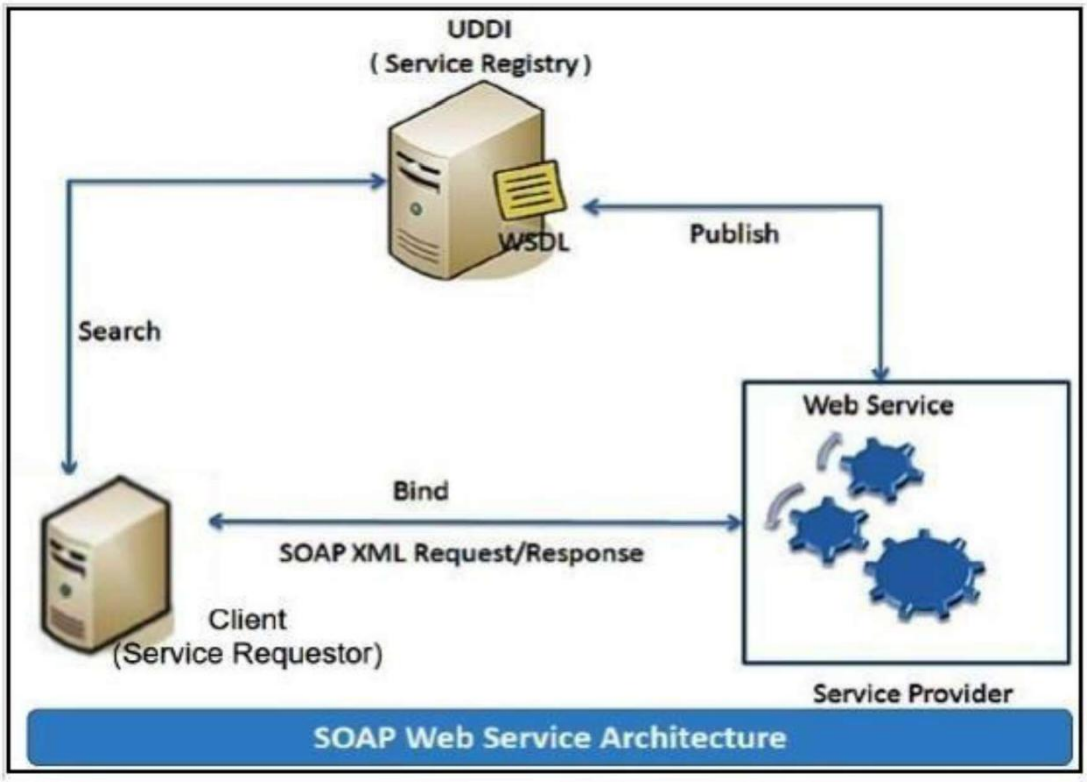
3. WSDL (Web Service Description Language) - XML-based file describing the service

SOAP web services:

Simple Object Access Protocol (SOAP) is an XML-based protocol for accessing web services. It is a W3C recommendation for communication between two applications, and it is a platform-and language-independent technology in integrated distributed applications.
While XML and HTTP together make the basic platform for web services, the following are the key components of standard SOAP web services:

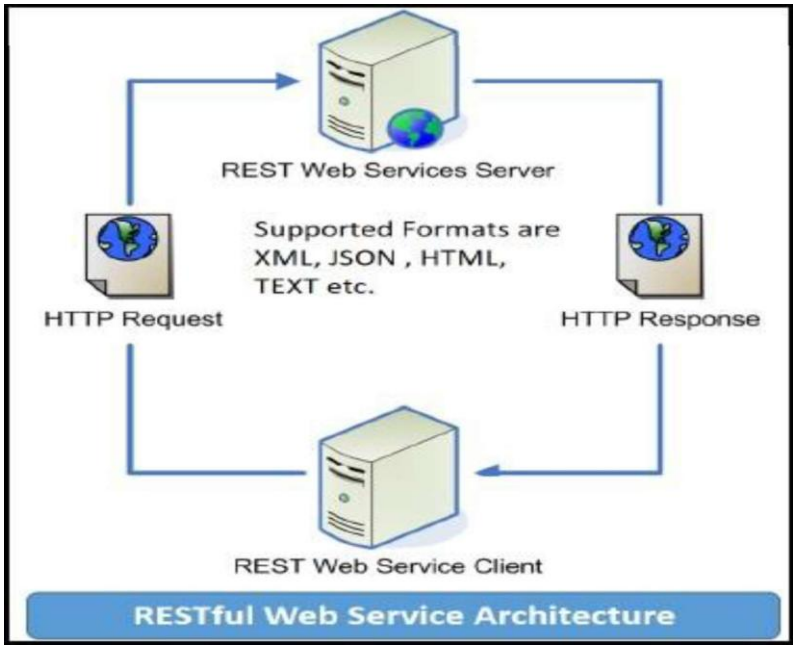
Universal Description, Discovery, and Integration (UDDI): UDDI is an XMLbased framework for describing, discovering, and integrating web services. It acts as a directory of webservice interfaces described in the WSDL language.

Web Services Description Language (WSDL): WSDL is an XML document containing information about web services, such as the method name, method parameters, and how to invoke the service. WSDL is part of the UDDI registry. It acts as an interface between applications that want to interact based on web services. The following diagram shows the interaction between the UDDI, Service Provider, and service consumer in SOAP web services:



RESTful web services

REST stands for Representational State Transfer. RESTful web services are considered a performance-efficient alternative to the SOAP web services. REST is an architectural style, not a protocol. Refer to the following diagram:



IMPLEMENTATION :

CalulatorClient.java

```
calculator > CalculatorClient.java > CalculatorClient
1   package calculator;
2   import javax.xml.namespace.QName;
3   import javax.xml.ws.Service;
4   import java.net.URL;
5
6   public class CalculatorClient {
        Run | Debug
7       public static void main(String[] args) throws Exception {
8           URL url = new URL(spec: "http://localhost:8080/calculator?wsdl");
9           QName qname = new QName(
10              namespaceURI: "http://calculator/",
11              localPart: "CalculatorServiceService"
12          );
13          Service service = Service.create(url, qname);
14          CalculatorInterface calc =
15              service.getPort(serviceEndpointInterface: CalculatorInterface.class);
16          int result = calc.add(a: 10, b: 20);
17          System.out.println("Result = " + result);
18      }
19  }
```

CalculatorInterface.java

```
alculator > CalculatorInterface.java > CalculatorInterface
1   package calculator;
2
3   import javax.jws.WebService;
4   import javax.jws.WebMethod;
5
6   @WebService
7   public interface CalculatorInterface {
8
9       @WebMethod
10      int add(int a, int b);
11  }
```

CalculatorPublisher.java

```java
package calculator;

import javax.xml.ws.Endpoint;
public class CalculatorPublisher {

    Run | Debug
    public static void main(String[] args) {
        Endpoint.publish(
            address: "http://localhost:8080/calculator",
            new CalculatorService()
        );
        System.out.println(x: "Web Service running at:");
        System.out.println(x: "http://localhost:8080/calculator?wsdl");
    }
}
```

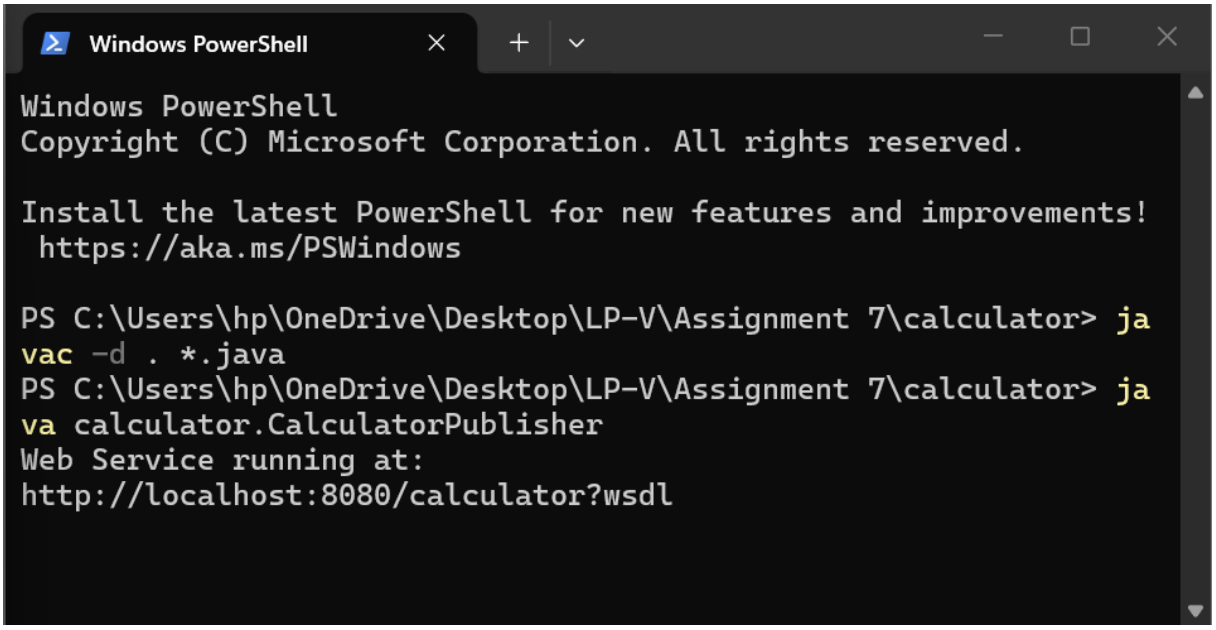
CalculatorService.java

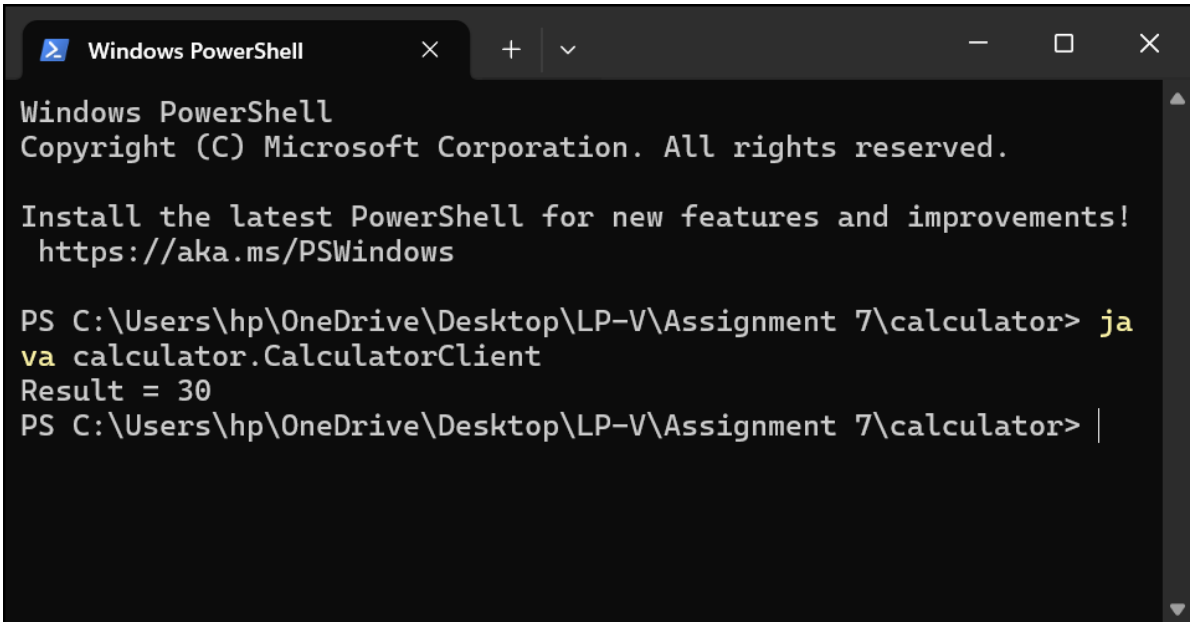
```java
package calculator;
import javax.jws.WebService;

@WebService(endpointInterface = "calculator.CalculatorInterface")
public class CalculatorService implements CalculatorInterface {

    @Override
    public int add(int a, int b) {
        return a + b;
    }
}
```

CONLCUSION :

In this experiment, we successfully implemented a SOAP-based web service using Java JAX-WS. The server was created to provide a service (addition operation), and the client was developed to consume this service over the network.

OUTPUT : -


Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.
Install the latest PowerShell for new features and improvements!
https://aka.ms/PSWindows
PS C:\Users\hp\OneDrive\Desktop\LP-V\Assignment 7\calculator> javac -d . *.java
PS C:\Users\hp\OneDrive\Desktop\LP-V\Assignment 7\calculator> java calculator.CalculatorPublisher
Web Service running at:
http://localhost:8080/calculator?wsdl


Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.
Install the latest PowerShell for new features and improvements!
https://aka.ms/PSWindows
PS C:\Users\hp\OneDrive\Desktop\LP-V\Assignment 7\calculator> java calculator.CalculatorClient
Result = 30
PS C:\Users\hp\OneDrive\Desktop\LP-V\Assignment 7\calculator>